

Fundamentals of API Development

Design Lab Discussion: Contract-First vs. Code-First

Day 2 - Session 3

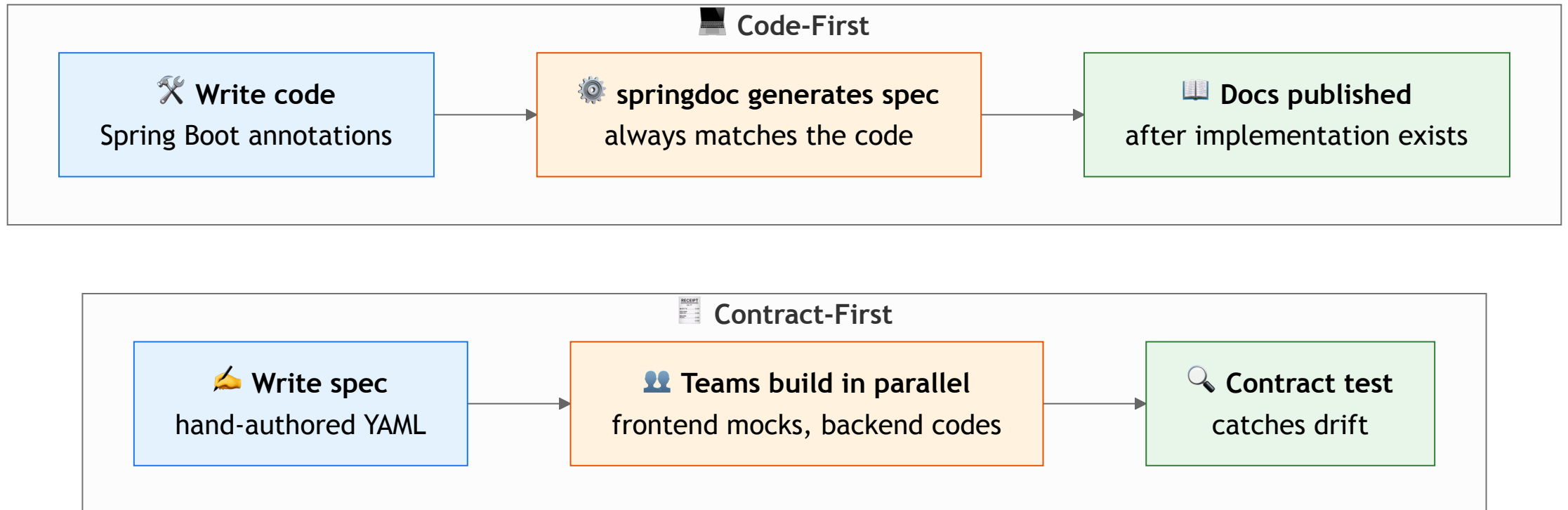
What We'll Cover

1. The two orders a spec and its code can come in
2. The tradeoff: parallelism against drift risk
3. Demo: consuming an API both ways in Postman
4. Group review of Lab 1.2 and Lab 1.3
5. Day 2 wrap-up and Day 3 preview

Two Orders, One Contract

Which comes first changes what you owe

Contract-First vs Code-First



Contract-First in Practice (Financial Services)

REAL-WORLD SCENARIO

Financial Services (Capital One — Contract-First Deployment):

- Capital One deploys API contracts to production within 48 hours of team alignment. The OpenAPI spec is the first artifact. Frontend and backend teams build in parallel against that deployed contract.
- UI engineers never wait for backend logic to be complete. The contract defines request and response shapes, and both sides implement against it simultaneously.

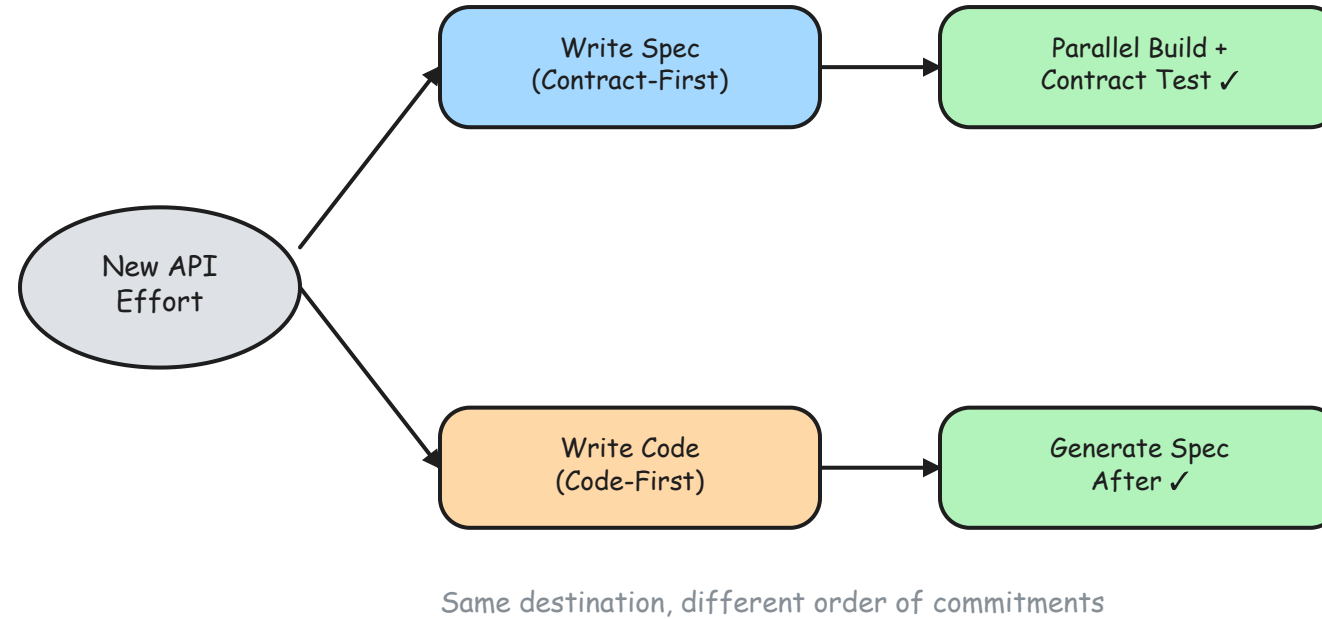
Contract-First in Practice (Retail & E-Commerce)

SKIP**REAL-WORLD SCENARIO**

Retail & E-Commerce (Walmart — Multi-Channel Solutions):

- Walmart's MCS team publishes the OpenAPI spec first. Partners then generate Java clients from it using OpenAPI Generator before Walmart's server implementation is complete.
- The spec is the contract. The code, both Walmart's server and partners' clients, implements it. No team waits for the other to finish.

A Fork in the Road



REFERENCE ONLY

The Tradeoff: Discussion

Your startup has a tight deadline. The frontend team is blocked waiting for the backend team to build the new API.

Should you use a Contract-First or a Code-First approach?

DISCUSS

How does the order of writing the spec impact team parallelism?

The Tradeoff: The Reality

	Contract-first	Code-first
Sequence	Spec first, code implements it	Code first, spec generated from it
Source of truth	The <code>.yaml</code> file	The running code
Parallelism	Teams build against the contract at once	Frontend waits for code or a stub
Drift risk	Needs contract testing to enforce	Can't drift; docs always match the code

The Tradeoff at Scale (Financial Services)

REAL-WORLD SCENARIO

Financial Services (Capital One — Central API Catalog):

- Capital One's contract-first approach at scale requires every microservice to register its API contract in a central catalog. Hundreds of services communicate through governed contracts.
- The payoff is consistency across the entire ecosystem. The cost is the discipline of maintaining that catalog and running contract tests to prevent drift.

The Tradeoff at Scale (Retail & E-Commerce)

SKIP**REAL-WORLD SCENARIO**

Retail & E-Commerce (Shopify — Code-First Admin REST API):

- Shopify's Admin REST API uses a code-first approach. The API evolves in the code, and the OpenAPI spec is generated. No hand-authored spec exists to drift.
- For a platform with rapid iteration across 200+ endpoints, code-first prevents spec drift by making the code the source of truth. The tradeoff is less upfront parallelism for third-party developers.

What 2026 Reporting Shows

Industry reporting describes contract-first as the practice that persisted: teams write the spec, then build against it in parallel, before server code exists.

NOTE

Roughly 82% of organizations report some level of API-first adoption, the same figure from Day 1.

Parallelism is the practical payoff; contract testing is what most teams add to catch the drift risk.

Code-First Still Has a Place

For a small team or an early API, hand-authoring a spec before any code exists is overhead without a second team waiting on it. springdoc-openapi's generated spec documents what already works and never drifts, because it is regenerated on every run.

Neither approach is universally correct. Match it to whether anyone is waiting on the contract.

When Code-First Fits (Financial Services)

REAL-WORLD SCENARIO

Financial Services (Navy Federal Credit Union — Internal Reporting API):

- A small NFCU team building a single-use reporting API for the credit risk group may not need a contract-first ceremony. springdoc generates the spec from the running code.
- The generated spec documents exactly what works. A hand-authored spec would be overhead with no second team waiting on the contract.

When Code-First Fits (Retail & E-Commerce)

SKIP**REAL-WORLD SCENARIO**

Retail & E-Commerce (Target — Prototype Recommendation Endpoint):

- A Target team prototyping a new personalized recommendation endpoint starts code-first. springdoc generates the spec from annotations as the endpoint takes shape.
- Once the endpoint stabilizes and the mobile app team needs to integrate, the generated spec becomes the contract for parallel work. Code-first got the prototype running; the spec enabled parallelism when it was needed.

Demo: A Mock Built Only from the Contract

From `day2/demos`, run `java -cp target/classes com.kavinschool.demos.DemoContractFirstMock`.

- Every response comes from the Lab 1.2 spec's examples alone
- No backend implementation exists yet
- A frontend team, or Postman, can build and test against this today

Offline, no API key, no network.

Demo: Code-First Docs Cannot Drift

From `day2/demos`, run `java -cp target/classes com.kavinschool.demos.DemoCodeFirstAlwaysInSync`.

- A springdoc-style registry regenerates its doc from the code on every run
- A new endpoint appears in the generated doc immediately, no doc edit made
- A hand-maintained snapshot, left untouched, silently falls behind

Offline, no API key, no network.

Demo: Consuming an API Both Ways

REFERENCE ONLY

Using Postman, hit two endpoints side by side.

- A mock server built purely from your Lab 1.2 spec (contract-first)
- The running quickstart API's springdoc-generated docs (code-first)

Both expose the same information. The difference is which one existed first, and which one wins when they disagree.

Group Review: Lab 1.2 and Lab 1.3 Outputs

Discuss with your group, using evidence from your own labs:

1. Where did your Lab 1.3 review catch something the spec author missed?
2. Which approach would have caught your Lab 1.1 findings earlier?
3. Would a contract test have caught the mismatch, if there was one?

Running behind? Use this as catch-up time for Lab 1.2/1.3.

Key Takeaways

1. Contract-first and code-first differ in sequence, not in which artifact ends up existing.
2. Contract-first buys parallelism at the cost of needing contract tests to prevent drift.
3. Code-first cannot drift, because the spec is regenerated from the code every time.
4. Postman consuming a mock vs a live API is the fastest way to make the distinction concrete.

Day 2 Wrap-Up

Next: Day 3 - API Lifecycle & Security

What You Have Now

From today	Feeds
A peer-reviewed OpenAPI spec with interactive docs	Day 3's versioning strategy needs a stable contract to version
Contract-first vs code-first tradeoffs	Day 3's lifecycle discussion, another sequencing tradeoff
springdoc-openapi and Swagger UI	Where security annotations will appear on Day 3

Day 3 Preview

Tomorrow your contract gets a lifecycle: versions, deprecation, and security rules.

1. API lifecycle overview and versioning
2. Lab 2.1: designing a non-breaking version strategy
3. Security for APIs: authentication, authorization, rate limiting
4. Lab 2.2: implementing basic API security rules

Bring your Lab 1.2 spec. Day 3 versions the resource you just designed.

Questions?

See you tomorrow